

TP 1 : Introduction au DevSecOps

Pipeline CI/CD Sécurisé

Travail Pratique N°1



Saad KORCHI

Étudiant en Cybersécurité

Encadré par :

Pr. AMAMOU Ahmed

EIDIA Cybersécurité

SOMMAIRE

1	Introduction	3
2	Objectifs pédagogiques	3
3	Architecture du Projet	3
3.1	Structure du projet	3
3.2	Composants de l'application	4
4	Pipeline CI/CD	4
4.1	Définition du pipeline	4
4.2	Fichier .gitlab-ci.yml	4
5	Démarche Pas à Pas	5
5.1	Étape 1 : Initialisation du projet	5
5.2	Étape 2 : Création du serveur avec upload	5
5.3	Étape 3 : Création des tests unitaires	5
6	Bilan de la première partie	6
	Captures d'écran	7

1 Introduction

Contexte du travail pratique

Ce rapport présente le Travail Pratique sur l'introduction au **DevSecOps** et aux pipelines **CI/CD sécurisés**. L'objectif est de mettre en place une chaîne d'intégration et de déploiement continu avec des vérifications de sécurité automatisées pour une application Node.js de type upload de fichiers.

2 Objectifs pédagogiques

Les objectifs de ce TP sont :

Comprendre les principes du **DevSecOps** et l'importance de la sécurité dans les pipelines CI/CD

Configurer un pipeline **CI/CD avec GitLab CI**

Intégrer des outils de sécurité dans le pipeline :

- **gitleaks** pour la détection de secrets
- **npm audit** pour les vulnérabilités des dépendances

Conteneuriser l'application avec **Docker**

Automatiser le build, les tests, la sécurité, le packaging et le déploiement

Comprendre le cycle de vie d'un pipeline **DevSecOps**

Shift Left

La sécurité est intégrée dès le début du cycle de développement

3 Architecture du Projet

3.1 Structure du projet

Voici l'arborescence complète du projet développé :

3.2 Composants de l'application

Composant	Description
Backend	Serveur Node.js avec Express (fichier <code>server.js</code>)
Frontend	Fichiers statiques dans le dossier <code>public/</code>
Uploads	Dossier pour les fichiers téléversés (<code>uploads/</code>)
Dépendances	Gérées via npm (<code>package.json</code>)
Conteneurisation	Docker avec <code>Dockerfile</code>
CI/CD	Pipeline GitLab avec <code>.gitlab-ci.yml</code>

4 Pipeline CI/CD

4.1 Définition du pipeline

Le pipeline est composé des **5 stages** suivants :



Stages du pipeline :

- **Build** : Installation des dépendances et vérification syntaxique
- **Test** : Exécution des tests unitaires
- **Security** : Scan de secrets et vulnérabilités
- **Package** : Construction de l'image Docker
- **Deploy** : Déploiement en staging/production

4.2 Fichier `.gitlab-ci.yml`

Extrait du fichier de configuration :

```

1 stages:
2   - build
3   - test
4   - security
5   - package
6   - deploy
7
8 variables:
9   APP_NAME: "upload-app"
10  APP_VERSION: "1.0.0"
11
12 build-job:
13   stage: build
14   image: node:18
15   script:
16     - npm install
17     - node -c server.js
  
```

```
18   artifacts:
19     paths:
20       - node_modules/
21     expire_in: 1h
22
23 test-job:
24   stage: test
25   image: node:18
26   script:
27     - npm test
28   coverage: '/All_files[- ]\*\[~\]+([\d\.]+)/'
```

5 Démarche Pas à Pas

5.1 Étape 1 : Initialisation du projet

```
1 mkdir upload-app
2 cd upload-app
3 npm init -y
4 npm install express multer
5 npm install --save-dev jest supertest
```

5.2 Étape 2 : Création du serveur avec upload

Fichier `server.js` avec :

- Configuration de `Multer` pour les uploads
- Dossier `uploads/` pour le stockage
- Routes GET/POST/DELETE

5.3 Étape 3 : Création des tests unitaires

```
1 test('upload de fichier', async () => {
2   const response = await request(app)
3     .post('/upload')
4     .attach('file', 'test.txt');
5   expect(response.statusCode).toBe(200);
6 });
```

6 Bilan de la première partie

Résumé des acquis

- Mise en place de la structure du projet Node.js
- Création du serveur Express avec gestion d'upload
- Écriture des tests unitaires avec Jest
- Configuration initiale du pipeline CI/CD GitLab

Fin du rapport TP1 - La suite sera traitée dans le TP2

Captures d'écran

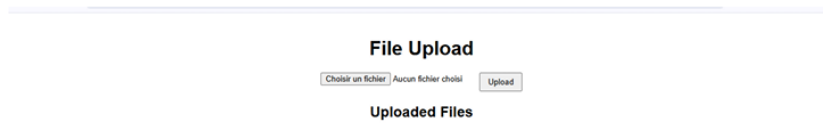


FIGURE 1 – Figure 1

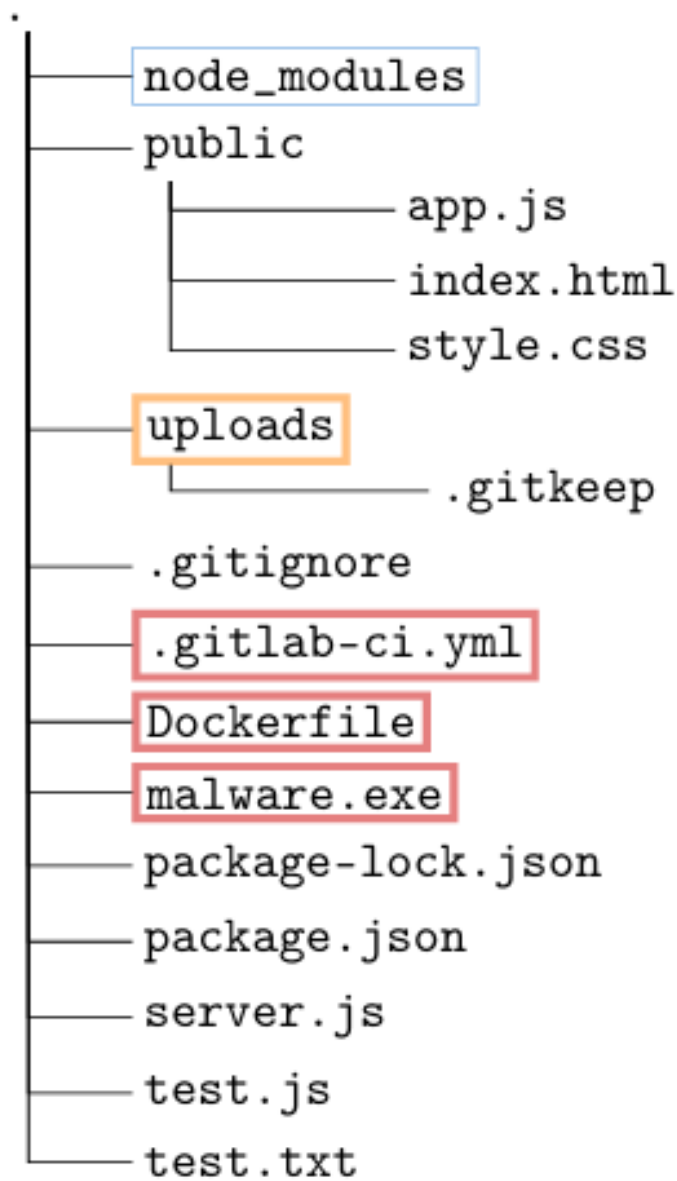


FIGURE 2 – Figure 2

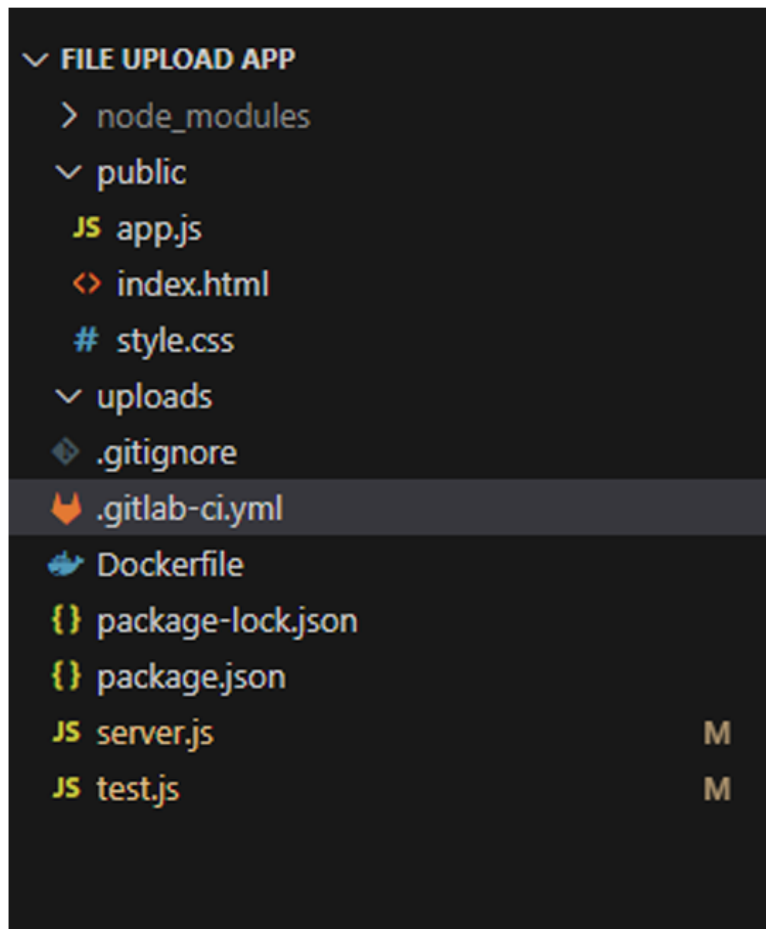


FIGURE 3 – Figure 3

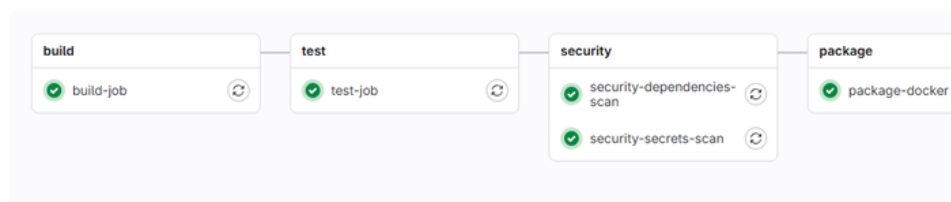


FIGURE 4 – Figure 4